Universidad Simón Bolívar

Departamento de Computación y Tecnología de la Información

CI3825 – Sistemas de Operación I

Trimestre Enero–Marzo 2026

Proyecto 2

Simulación Concurrente de un Sistema de Colas Bancario

1. Objetivo

Diseñar e implementar en C una simulación concurrente de un sistema de atención bancaria utilizando hilos POSIX (pthreads). El proyecto evalúa prioritariamente las siguientes competencias:

- Uso correcto de concurrencia con pthreads
- Sincronización mediante mutex (pthread_mutex_t)
- Ausencia de condiciones de carrera (race conditions)
- Prevención y ausencia de deadlocks
- Terminación correcta y ordenada de hilos

La teoría de colas (modelo M/M/c) se utiliza únicamente como herramienta de validación cuantitativa de los resultados de la simulación.

2. Lenguaje y Entorno de Desarrollo

Toda la implementación debe realizarse en C, respetando los siguientes requisitos de compilación y entorno:

- Lenguaje: C (estándar C11, flag -std=c11)
- Compilador: gcc (versión ≥ 9)
- Plataforma: GNU/Linux (kernel ≥ 4.x)
- Biblioteca de hilos: POSIX Threads (pthreads), enlazar con -lpthread
- Biblioteca matemática: enlazar con -lm (para exp(), log())
- Se prohíbe el uso de cualquier otro lenguaje de programación
- Se prohíbe el uso de hilos de C++ (std::thread) u otras abstracciones no POSIX

Flags de compilación obligatorios:

```
gcc -std=c11 -Wall -Wextra -Werror -pthread -lm -o banco banco.c
```

El código debe compilar sin warnings ni errores con los flags anteriores. La entrega que no compile no será evaluada.

3. Modelo Temporal

El tiempo se modela en segundos lógicos (no en tiempo real del sistema). El banco opera bajo las siguientes reglas temporales:

- El banco abre en t = 0.
- El banco cierra en t = T_{cierre} (definido en banco.txt).

- No se aceptan clientes cuyo tiempo de llegada sea ≥ T_{cierre}.
- El banco continúa operando hasta que el último cliente haya sido atendido completamente, incluso si ese momento es posterior a T_{cierre}.

4. Archivo de Configuración

El programa debe leer obligatoriamente un archivo de configuración .txt ubicado en el mismo directorio del ejecutable. Este archivo de configuración .txt se le pasará al ejecutable como único parámetro cuando se ejecuta este:

```
./banco modelo1.txt
```

4.1 Reglas de Sintaxis

- Todos los nombres de parámetros deben estar en MAYÚSCULAS.
- Los comentarios se inician con # y se extienden hasta el fin de línea.
- Las líneas en blanco deben ignorarse.
- El formato obligatorio de cada parámetro es: PARAMETRO=VALOR (sin espacios alrededor del =).
- El orden de los parámetros en el archivo es libre.

4.2 Parámetros Requeridos

```
# Archivo de configuración banco.txt
# Comentarios permitidos con #

CAJEROS=4
TCIERRE=1000
LAMBDA=0.05
MU=0.08
MAX_CLIENTES=300
```

4.3 Definiciones de Parámetros

Parámetro	Tipo	Descripción	Restricción
CAJEROS	int	Número de hilos servidores (cajeros)	≥ 1
TCIERRE	int	Tiempo de cierre del banco (segundos lógicos)	> 0
LAMBDA (λ)	double	Tasa promedio de llegada (clientes/segundo)	> 0
MU (μ)	double	Tasa promedio de servicio (clientes/segundo)	> 0
MAX_CLIENTES	int	Límite superior para truncar clientes generados. Si N* > MAX_CLIENTES, el resumen final debe indicarlo con la línea: "Clientes truncados por MAX_CLIENTES: SI"	≥ 1

4.4 Manejo de Errores en el Archivo de Configuración

El programa debe validar el archivo de configuración al inicio y terminar con un mensaje de error descriptivo en stderr si:

- El archivo .txt no existe o no puede abrirse.
- Alguno de los cinco parámetros requeridos está ausente.
- Algún valor no es del tipo correcto (e.g. texto donde se espera un número).
- Algún valor viola su restricción (e.g. CAJEROS = 0, LAMBDA = -1.0).
- Hay un parámetro desconocido (opcional: puede emitir un warning y continuar).

No existen valores por defecto: todos los parámetros son obligatorios.

5. Modelo Matemático (M/M/c)

Se modela un sistema M/M/c donde las llegadas siguen un proceso de Poisson y los tiempos de servicio son variables aleatorias exponenciales independientes. El número de servidores en paralelo es c = CAJEROS.

5.1 Número de Clientes

La propiedad fundamental del proceso de Poisson es que los tiempos entre llegadas consecutivas son variables aleatorias exponenciales independientes. Esto significa que no hace falta samplear N de forma separada: basta con generar tiempos inter-llegada exponenciales acumulados y detener el proceso cuando el acumulado supere TCIERRE. El número de clientes N resulta naturalmente de ese proceso, y es equivalente a haber sampleado una variable Poisson(λ × TCIERRE). El procedimiento de implementación es el siguiente:

```
acumulado = 0.0;  N = 0;
while (N < MAX_CLIENTES) {
    acumulado += -log(U) / LAMBDA;   /* T_i ~ Exp(λ) */
    if (acumulado >= TCIERRE) break;
    cliente[N].llegada = acumulado;
    N++;
}
```

Al salir del bucle, N contiene el número total de clientes que llegaron antes de TCIERRE, y cliente[i].llegada ya tiene el tiempo de llegada A_i de cada uno. No es necesario ningún paso adicional de descarte ni de generación separada de N. El límite MAX_CLIENTES actúa como tope de seguridad; si se alcanza, el programa debe indicarlo en la salida (ver sección 11).

5.2 Generación de Tiempos de Llegada

La fórmula para cada tiempo inter-llegada, usada directamente dentro del bucle de generación de la sección 5.1, es:

T_i ~ Exp(λ) → $T_i = \frac{-\ln(U)}{\lambda}$, donde U ~ Uniforme(0,1)

Nota: A_i es exactamente el valor de la variable acumulado en el momento en que se registra el cliente i dentro del bucle de la sección 5.1. No es necesario recalcularlo.
El bucle termina en cuanto el acumulado supera TCIERRE, por lo que todos los clientes registrados tienen automáticamente $A_i < T_{cierre}$.

5.3 Tiempo de Servicio

S_i ~ Exp(μ) → $S_i = \frac{-\ln(U)}{\mu}$, donde U ~ Uniforme(0,1)

La generación de números pseudoaleatorios debe realizarse con las funciones rand() / srand() de la biblioteca estándar de C, inicializando la semilla con srand(time(NULL)) al comienzo del programa. Dado que la simulación es estocástica, los resultados varían entre ejecuciones. Para la evaluación se considerarán válidos errores relativos Wq y W menores al 15% cuando N ≥ 100 clientes; con parámetros que generen menos clientes, los errores pueden ser mayores y se valorará la correctitud del método, no la convergencia exacta.

5.4 Fórmulas de Erlang-C (Modelo M/M/c)

Con c = CAJEROS, λ = LAMBDA y μ = MU, se definen:

Tráfico ofrecido	$a = \frac{\lambda}{\mu}$

Utilización por servidor	$\rho = \frac{a}{c}$
Condición de estabilidad	$\rho < 1$
Factor de Erlang-C (probabilidad de espera)	$C(c,a) = \frac{\frac{a^c}{c!\ (1-\rho)}}{\sum_{k=0}^{c-1} \frac{a^k}{k!} + \frac{a^c}{c!\ (1-\rho)}}$
Tiempo promedio de espera en cola (teórico)	$W_{q,teo} = \frac{C(c,a)}{c\mu - \lambda}$
Tiempo promedio en el sistema (teórico)	$W_{teo} = W_{q,teo} + \frac{1}{\mu}$

Si $\rho \geq 1$, el programa debe mostrar una advertencia y omitir el cálculo de métricas teóricas.

Nota de implementación:

- El sumatorio $\sum_{k=0}^{c-1} a^k/k!$ y el término $a^c/c!$ deben calcularse iterativamente para evitar desbordamiento numérico.
- Se recomienda acumular los términos en orden creciente de k.
- Para valores grandes de c, usar log-factorial o funciones de precisión extendida evita errores de overflow.

6. Arquitectura Concurrente

Esta sección es la más importante del proyecto.

6.1 Estructura General

- Una cola compartida FIFO de clientes (estructura de datos a definir por el estudiante).
- Exactamente CAJEROS hilos de tipo pthread_t, cada uno representando un cajero.
- Un mutex (pthread_mutex_t) para proteger todos los accesos a la cola y a las estadísticas globales.
- Una variable de condición (pthread_cond_t) para que los hilos esperen eficientemente cuando la cola esté vacía.
- Una estructura global de estadísticas compartida protegida por el mismo mutex o por uno dedicado.

6.2 Uso Obligatorio de pthreads

```
/* Creación de hilos */
pthread_t cajeros[CAJEROS];
for (int i = 0; i < CAJEROS; i++)
    pthread_create(&cajeros[i], NULL, funcion_cajero, arg);

/* Espera de finalización */
for (int i = 0; i < CAJEROS; i++)
    pthread_join(cajeros[i], NULL);
```

No se permite ninguna solución secuencial disfrazada de concurrente (e.g. crear hilos que no ejecuten trabajo real en paralelo).

6.3 Secciones Críticas Obligatorias

Las siguientes operaciones DEBEN estar protegidas con pthread_mutex_lock / pthread_mutex_unlock:

- Inserción de clientes en la cola.
- Extracción (dequeue) de clientes de la cola.
- Actualización de contadores globales (clientes atendidos, etc.).
- Actualización del tiempo máximo de espera.
- Registro de cualquier métrica compartida (Wq, W, etc.).
- Impresión de eventos (recomendable protegerla para evitar intercalado).

6.4 Prevención de Condiciones de Carrera

El programa no debe:

- Atender un mismo cliente más de una vez.
- Perder clientes (el total de clientes atendidos debe igualar al generado).
- Generar inconsistencias estadísticas (e.g. suma de esperas incorrecta).
- Acceder a datos compartidos fuera de una sección crítica.

6.5 Prevención de Deadlocks

Deben satisfacerse las siguientes reglas:

- No anidar locks: nunca llamar a pthread_mutex_lock mientras se sostiene el mismo mutex.
- Liberar siempre el mutex: incluso en rutas de error o return temprano.
- Evitar la espera circular: si se usan múltiples mutex, adquirirlos siempre en el mismo orden global.
- Las secciones críticas deben ser lo más cortas posible (sección crítica mínima).

6.6 Terminación Correcta de los Hilos

El mecanismo de terminación debe funcionar de la siguiente manera:

- El hilo principal (main) genera todos los clientes, los encola en orden de llegada y luego lanza los hilos cajeros. No se usa timing real: todos los clientes se insertan en la cola antes de que comiencen las atenciones.
- Una variable global banco_cerrado indica que no llegarán más clientes.
- Cada hilo cajero verifica: si la cola está vacía y banco_cerrado == 1, termina.
- Se usa pthread_cond_broadcast para despertar a todos los hilos cuando se cierra el banco.
- main llama a pthread_join sobre cada hilo antes de imprimir el resumen final.

```
/* Secuencia de cierre (después de encolar todos los clientes y lanzar los hilos) */
pthread_mutex_lock(&mutex_cola);
banco_cerrado = 1;
pthread_cond_broadcast(&cond_cola);   /* despertar hilos bloqueados */
pthread_mutex_unlock(&mutex_cola);

for (int i = 0; i < CAJEROS; i++)
    pthread_join(cajeros[i], NULL);
```

7. Tiempo Lógico vs. Tiempo Real

El proyecto distingue estrictamente entre dos nociones de tiempo:

Concepto	Descripción	Función en C
Tiempo real (wall-clock)	Tiempo que tarda el proceso en ejecutarse en el sistema operativo.	time(), clock_gettime()
Tiempo lógico de simulación	Tiempo matemático calculado a partir de A_i, B_i y F_i de cada cliente.	Variables double en el código

IMPORTANTE: la simulación debe basarse exclusivamente en tiempo lógico. Está prohibido usar sleep(), usleep() o cualquier función de pausa real para simular el paso del tiempo. Todos los timestamps impresos corresponden al tiempo lógico.

El tiempo de inicio de atención de un cliente por el cajero i se calcula como:

```
B_i = max(A_i, Fi_anterior_del_cajero_i)

Donde Fi_anterior es el tiempo en que el cajero i terminó de atender
al cliente previo (o 0.0 si el cajero no ha atendido ningún cliente). Cada hilo cajero debe
mantener su propia variable local tiempo_libre, inicializada en 0.0 y actualizada a F_i tras cada
atención.
```

8. Eventos Durante la Simulación

El programa debe imprimir en stdout un registro de eventos con el siguiente formato obligatorio. Los cajeros se numeran del 1 al CAJEROS según el índice del array pthread_t cajeros[] (el hilo cajeros[0] es "Cajero 1", cajeros[1] es "Cajero 2", etc.); esta numeración es fija durante toda la ejecución.

```
[t=XX.XX] DESCRIPCION_DEL_EVENTO
```

8.1 Tipos de Eventos Obligatorios

```
[t=45.23] Cliente 7 llega al banco
[t=47.10] Cliente 7 inicia atención en Cajero 2
[t=52.85] Cliente 7 finaliza atención en Cajero 2
```

8.2 Ejemplo Parcial de Salida

```
[t= 3.12] Cliente 1 llega al banco
[t= 3.12] Cliente 1 inicia atención en Cajero 1
[t= 5.47] Cliente 2 llega al banco
[t= 8.10] Cliente 1 finaliza atención en Cajero 1
[t= 8.10] Cliente 2 inicia atención en Cajero 1
[t=12.35] Cliente 3 llega al banco
[t=14.88] Cliente 2 finaliza atención en Cajero 1
[t=14.88] Cliente 3 inicia atención en Cajero 1
```

Nota: el orden de impresión puede variar por efectos de la concurrencia, pero los timestamps individuales de cada cliente deben ser coherentes ($A_i \leq B_i \leq F_i$ para todo cliente i).

9. Métricas Simuladas

Para cada cliente i se definen los siguientes tiempos lógicos:

Variable	Fórmula	Significado
A_i		Tiempo de llegada al banco
B_i	max(Ai, Fant_cajero)	Tiempo de inicio de atención
F_i	B_i+S_i	Tiempo de fin de atención
$W_{q,i}$	B_i-A_i	Tiempo de espera en cola
W_i	F_i-A_i	Tiempo total en el sistema

Métricas globales calculadas al finalizar la simulación:

```
Wq_sim  = (1/N) · Σ Wq,i          /* promedio de espera en cola */
W_sim   = (1/N) · Σ Wi            /* promedio de tiempo en sistema */
Wq_max  = max(Wq,i)               /* tiempo máximo de espera */
T_total = max(Fi)                 /* tiempo hasta que el último cliente termina */
```

10. Comparación Teórica (Modelo M/M/c)

Al finalizar la simulación, el programa debe calcular las métricas teóricas usando las fórmulas de Erlang-C descritas en la sección 5.4 y compararlas con los resultados simulados:

```
Error relativo Wq = |Wq_sim - Wq_teo| / Wq_teo * 100  [%]
Error relativo W  = |W_sim  - W_teo | / W_teo  * 100  [%]
```

Si ρ ≥ 1, el programa debe indicar que el sistema es inestable y omitir el cálculo de métricas teóricas y errores relativos.

11. Salida Final Obligatoria

Al finalizar la simulación, el programa debe imprimir el siguiente resumen en stdout:

```
==================================================
                 RESUMEN FINAL
==================================================
Parametros:
  CAJEROS:       4
  TCIERRE:       1000
  LAMBDA:        0.05
  MU:            0.08
  MAX_CLIENTES:  300
Resultados Simulados:
  Clientes atendidos:                187
  Truncado por MAX_CLIENTES:        NO
  Tiempo promedio de espera (Wq):    12.45
  Tiempo promedio en sistema (W):    19.32
  Tiempo maximo de espera:           41.77
  Tiempo total hasta ultimo cliente: 1089.34
Resultados Teoricos (M/M/c):
  Utilizacion (rho):                 0.1563
  Tiempo promedio de espera teorico: 13.02
  Tiempo promedio en sistema teorico: 19.50
  Error relativo Wq: 4.3 %
  Error relativo W:  0.9 %
Estado del sistema:
  rho = 0.1563 < 1  →  Sistema estable
==================================================
```

12. Estructura del Proyecto y Entrega

El proyecto debe entregarse como un archivo comprimido con el siguiente formato:

```
Proy2_CARNET1_CARNET2.tar.xz

Ejemplo:  Proy2_12-34567_98-76543.tar.xz
```

12.1 Contenido Obligatorio del Archivo

```
Proy2_CARNET1_CARNET2/
├── banco.c              ← código fuente principal
├── banco.h              ← cabecera con tipos y prototipos (si aplica)
├── banco.txt            ← archivo de configuración de ejemplo
├── Makefile             ← reglas de compilación
├── informe.pdf          ← informe del proyecto (máx. 4 páginas)
└── declaracion.pdf      ← declaración de autenticidad firmada
```

12.2 Makefile Mínimo

El Makefile debe incluir al menos los targets:

```
CC      = gcc
CFLAGS  = -std=c11 -Wall -Wextra -Werror -pthread
LDFLAGS = -lm

all: banco

banco: banco.c
   $(CC) $(CFLAGS) -o banco banco.c $(LDFLAGS)

clean:
   rm -f banco
```

12.3 Ejecución del Programa

```
# El archivo banco.txt debe estar en el mismo directorio
./banco banco.txt
```

12.4 Fecha y Plataforma de Entrega

- Fecha límite: lunes 09 de marzo de 2026, antes de las 8:30 A.M., por Google Classroom.
- El archivo tar.xz debe crearse con: tar cJf Proy2_X_Y.tar.xz Proy2_X_Y/
- No se aceptan entregas tardías ni por correo electrónico.

13. Informe del Proyecto

El informe es un documento PDF de máximo 4 páginas en total (portada + máx. 3 páginas de cuerpo + referencias) que debe incluir las siguientes secciones:

13.1 Portada

- Nombre, apellido y carné de cada integrante del equipo.
- Nombre del curso y trimestre.
- Título del proyecto.

13.2 Cuerpo del Informe (máx. 3 páginas, sin contar portada ni referencias)

El cuerpo debe abordar obligatoriamente los siguientes puntos (puede organizarlos en subsecciones):

- Diseño concurrente: descripción de la arquitectura de hilos, la cola compartida y los mecanismos de sincronización usados.
- Identificación de secciones críticas: qué datos comparten los hilos y por qué es necesario protegerlos.
- Justificación de la sincronización: por qué se eligió el esquema de mutex y variable de condición utilizado.

- Estrategia para evitar deadlocks: cómo se garantiza que no ocurran ciclos de espera.
- Comparación teórica vs. simulación: análisis de los errores relativos obtenidos y posibles causas de discrepancia.

13.3 Referencias Bibliográficas

Listado de fuentes consultadas (libros, artículos, páginas web). Las referencias no cuentan dentro del límite de 3 páginas del cuerpo.

14. Criterios de Evaluación

La evaluación se realizará ejecutando el programa con distintos archivos .txt y revisando el código fuente. La distribución de puntaje es:

Criterio	Puntos (sobre 15)	Descripción
Concurrencia y sincronización	5	Creación correcta de hilos, uso adecuado de mutex, ausencia de race conditions y deadlocks.
Correctitud del modelo M/M/c	4	Generación exponencial correcta, fórmulas de Erlang-C implementadas, resultados coherentes.
Terminación y gestión de hilos	3	Todos los hilos finalizan correctamente con pthread_join y el programa termina sin errores.
Calidad del código e informe	3	Modularidad, legibilidad, comentarios, y presentación del informe con análisis teórico.

14.1 Causas de Nota Cero

- El código no compila con los flags obligatorios.
- La solución es secuencial (no usa hilos de forma real).
- El programa presenta comportamiento no determinista irreproducible atribuible a race conditions no resueltas.
- Se detecta plagio entre grupos.

15. Condiciones Generales

- No se permite la colaboración entre grupos distintos.
- El código y el informe deben ser de autoría propia
- Se prohíbe el uso de código generado por IA.
- La declaración de autenticidad debe estar firmada por todos los integrantes.
- Dudas sobre el enunciado deben plantearse antes del lunes 02 de marzo.

16. Referencias Útiles

- Silberschatz, Galvin & Gagne. Operating System Concepts, 10th ed. Capítulos 4 y 6.
- Kerrisk, M. The Linux Programming Interface. Capítulo 30 (Threads).
- man 7 pthreads — Documentación de POSIX Threads en Linux.
- Kleinrock, L. Queueing Systems, Vol. 1. Wiley, 1975. (Modelo M/M/c y Erlang-C).
- GNU C Library Manual: https://www.gnu.org/software/libc/manual/